WEB FORGE AI / SYSTEM BLUEPRINT

Jury Scoring Website

Architecture, database design, security model, and implementation plan

A simple scoring portal for two judges and one administrator.

STACK

HTML · CSS · JavaScript / Python Flask / Dedicated Supabase project

VISUAL DIRECTION

Black · Grey · Peach · White
Minimal, readable, sharp-edged. No rounded cards. No glassmorphism.

PURPOSE

Build a dependable jury portal for WEB FORGE AI. Judges score teams against five criteria in either the Machine Learning Model round or Web Development round. The Admin manages accounts, teams, criteria, evaluation status, and exports. Data lives in a dedicated Supabase project, separate from the quiz website.

01 / Product scope

Three account types, two evaluation rounds, and a small set of workflows. The backend is authoritative for identity, permissions, validation, and saved marks.

ROLE	CAN DO	CANNOT DO
Judge 1	View teams; score both rounds; save drafts; submit and check own status.	View or change Judge 2 marks; manage criteria, accounts, or teams.
Judge 2	Same workflow as Judge 1, with independently attributed scores.	View or change Judge 1 marks; access Admin functions.
Admin	Manage judges, teams, criteria, max marks/weights; inspect results; reopen evaluations; export CSV.	Admin actions should be auditable. Judge scoring is not required by default.

In scope

- Login/logout for Judge 1, Judge 2, and Admin.
- Team selection and criterion-based scoring.
- Draft save, submit, locking, and Admin-controlled reopening.
- Separate judge totals, combined results, incomplete status, and CSV export.
- Editable criteria, descriptions, order, maximum marks, and optional weights.

Out of scope for v1

No live quiz engine, Socket.IO, public registration, complex analytics, or frontend framework. Avoid adding features unless the event workflow requires them.

Database design decision

Use one shared **scores** table with a judge ID on every row rather than separate physical tables per judge. This keeps marks logically separated, makes judge names editable, and avoids schema changes when accounts change.

02 / Judging criteria

The criteria below are taken from the supplied screenshots. Numerical weights and maximum marks are not clearly visible, so the Admin must configure them. Do not invent a marking scheme.

Round A / Machine Learning (ML) Model

CRITERION	EVALUATION FOCUS	MAX / WEIGHT
Model Accuracy & Performance	Accuracy, precision, recall, F1-score, or other relevant metrics.	Admin-configured
Data Handling & Preprocessing	Data cleaning, feature selection, and dataset quality.	Admin-configured
Model Selection & Implementation	Algorithm choice, training approach, and technical implementation.	Admin-configured
Innovation & Problem-Solving	Originality, relevance, and effectiveness of the proposed solution.	Admin-configured
Presentation & Explanation	Clarity of approach, results, limitations, and live demonstration.	Admin-configured

Round B / Web Development

CRITERION	EVALUATION FOCUS	MAX / WEIGHT
UI/UX Design	Visual appeal, layout, navigation, and user experience.	Admin-configured
Functionality	Working features, interactions, and completion of requirements.	Admin-configured
Code Quality & Structure	Clean code, proper structure, maintainability, and best practices.	Admin-configured
Innovation & Creativity	Uniqueness, creativity, and effective problem-solving.	Admin-configured
Responsiveness & Performance	Mobile compatibility, loading speed, and overall performance.	Admin-configured

Configuration rule: Store criteria as database rows with round, description, order, active status, maximum marks, optional weight, and version. Freeze or version criteria once scoring starts so historical evaluations do not silently change.

03 / Application architecture

A three-layer design keeps the browser simple and makes the backend responsible for trusted decisions.

LAYER	RESPONSIBILITIES	IMPLEMENTATION
Frontend	Login, role-aware navigation, team list, score form, Admin forms, results, CSV trigger.	Plain HTML, CSS, JavaScript; fetch() calls.
Backend	Authentication, role checks, validation, locking, totals, audit logging.	Python Flask; modular routes and services.
Database	Users/profiles, teams, rounds, criteria, marks, evaluation status, audit records.	Dedicated Supabase PostgreSQL project.
Hosting	Serve static files and run the Flask service.	Static frontend host + Python-compatible backend host; HTTPS.

Request flow

1. User logs in through the frontend.
2. Flask verifies identity and creates/validates a secure session.
3. Frontend requests active teams, rounds, and criteria.
4. Judge saves a draft or submits an evaluation.
5. Flask checks role, judge identity, mark ranges, criterion/round relationships, and status.
6. Flask persists records in Supabase and returns success or a clear error.
7. Admin retrieves aggregate results and exports CSV.

Deployment rule

Keep the Supabase service-role key on the backend only. Never place it in HTML, JavaScript, public build variables, or source control. Configure exact-origin CORS, HTTPS, secure cookies, and production secrets.

04 / Database schema

Use a dedicated Supabase project, separate from the quiz website. Use UUID primary keys and foreign keys.

TABLE	CORE FIELDS	PURPOSE / RULES
judges / profiles	id, auth_user_id or username, display_name, role, is_active, created_at	Prefer Supabase Auth. Otherwise use a proven password-hashing library. Never store plain-text passwords.
participants	id, team_code, team_name, is_active, created_at	Admin manages teams; team_code should be unique.
rounds	id, name, display_order, is_active	Seed with ML Model and Web Development.
criteria	id, round_id, name, description, max_marks, weight, display_order, is_active, version	Numeric marks/weights; decide whether max_marks or weighted score is authoritative.
scores	id, judge_id, participant_id, round_id, criterion_id, marks, remarks, updated_at	Unique judge + participant + criterion; validate criterion belongs to round.
evaluation_status	id, judge_id, participant_id, round_id, status, submitted_at, locked_at	Unique judge + participant + round. Status: draft, submitted, locked.
audit_log	id, actor_id, action, entity_type, entity_id, metadata, created_at	Record Admin changes, reopening, and criteria edits.

Scoring rules

- Reject negative, non-numeric, and above-maximum marks.
- A missing score is incomplete, not zero.
- Decide whether the final result is the sum or average of both judges and define how weights apply.
- Version/freeze criteria after scoring begins. Deactivate old records rather than deleting history.
- Use database transactions where a multi-row submission must be all-or-nothing.

05 / Authentication, security, and API

Recommended authentication: Supabase Auth with a protected profile/role table, or a carefully implemented Flask session. The backend must authorize every protected route.

CONTROL	REQUIREMENT
Role enforcement	Check role on the server for every protected endpoint. Never trust a role sent by the browser.
Judge isolation	Take judge identity from the authenticated session/token, not from a request field. Judges edit only their own marks.
Session security	HTTPS; HttpOnly/Secure cookies where applicable; SameSite policy; expiry; logout invalidation; CSRF protection for cookie-authenticated mutations.
Database access	Keep privileged credentials server-side; least privilege; appropriate RLS policies; test unauthorized direct/API access.
Validation	Validate IDs, relationships, ranges, remarks length, and evaluation status in Flask.
Audit / abuse	Log Admin changes and reopen actions; rate-limit login attempts; never log passwords or tokens.

Initial API contract

METHOD	ENDPOINT	ACCESS	PURPOSE
POST	/api/login	Public	Authenticate
POST	/api/logout	Authenticated	End session
GET	/api/me	Authenticated	Identity and role
GET	/api/rounds, /criteria, /participants	Authenticated	Load event configuration
GET	/api/evaluations/<round>/<participant>	Judge/Admin	Load authorized evaluation
PUT	/api/evaluations/draft	Judge	Save own draft
POST	/api/evaluations/submit	Judge	Validate and submit
GET	/api/admin/results, /export	Admin	Results and CSV
POST/PATCH	/api/admin/judges, /criteria, /participants	Admin	Manage event setup

06 / Visual and interaction specification

The interface should feel editorial and precise, not like a glossy dashboard template. Use a strict grid, square corners, flat surfaces, and restrained peach accents.

TOKEN	HEX	USAGE
Ink black	#171717	Navigation, primary text, headers, primary buttons.
Charcoal grey	#414141	Secondary text and inactive states.
Muted grey	#777777	Hints, timestamps, helper copy.
Peach	#F2B99F	Accent, active state, focus underline, progress.
Pale peach	#FBE9E0	Selected row or restrained callout.
White	#FFFFFF	Main page and form surfaces.
Soft grey	#F4F3F1	Alternating table rows and secondary surfaces.

Non-negotiable design rules

• No rounded borders, rounded cards, or pill-shaped buttons. Use square corners and 1 px dividers.
• No glassmorphism, blur, translucent floating panels, or heavy gradients.
• Use peach sparingly for emphasis, not as a large decorative background.
• Put labels above inputs; include keyboard navigation and visible focus states.
• Scoring page shows team, round, five criteria, descriptions, maximum marks, inputs, live subtotal, save state, and submit action.
• Show unsaved-change warnings and confirm final submission.
• Keep the layout usable on laptop and mobile.

Core screens

SCREEN	ESSENTIAL CONTENT
Login	WEB FORGE AI wordmark, username/password, clear errors, no public registration.
Judge dashboard	Judge name, round selector, searchable team list, Draft/Submitted status, continue action.
Scoring form	Five dynamic criteria; max marks; mark inputs; subtotal; save draft and submit.
Admin dashboard	Compact navigation for Judges, Teams, Rounds & Criteria, Results. Prefer tables over decorative cards.
Results	Team, round, Judge 1 total, Judge 2 total, combined/average total, completion status, CSV export.

07 / Implementation action plan

Build in small, verifiable milestones. Each phase should produce a testable result before the next begins.

PHASE	WORK	ACCEPTANCE CHECK
1. Lock requirements	Confirm max marks/weights, sum vs average, team list, who judges which teams, draft and submission rules.	Written scoring specification approved.
2. Supabase setup	Create separate project, tables, foreign keys, unique constraints, indexes, Auth/profile roles, and RLS.	Test access follows intended permissions.
3. Flask foundation	Configuration, secrets, error handling, auth middleware, role checks, database service layer.	Anonymous requests cannot access protected routes.
4. Judge workflow	Dashboard, round/team selection, dynamic criteria form, draft save, validation, submit/lock.	Both judges can complete an evaluation; scores persist.
5. Admin workflow	CRUD for judges, participants, criteria, and rounds; deactivate rather than delete history.	Admin changes appear without redeploying frontend.
6. Results/export	Agreed totals formula, incomplete-entry reporting, judge comparison, CSV export.	Displayed and exported totals match hand calculations.
7. Security tests	Cross-judge edits, role escalation, over-max scores, duplicates, locked edits, session/CSRF tests.	Unauthorized requests fail and sensitive actions are auditable.
8. Deployment	Deploy frontend and Flask, configure HTTPS, CORS, secrets, logging, backups, health check.	End-to-end test passes from a separate browser/device.

Suggested delivery order

First make login and one test team work. Then load criteria from Supabase, add score persistence, build Admin management, and finish with reports, security tests, and deployment. This keeps the first working version small and limits debugging complexity.

Definition of done

A judge can log in, evaluate a team in either round, save and submit marks, and only access their own scores. Admin can manage event configuration, inspect both judges' results, reopen an evaluation with an audit trail, and export accurate results. The site works on desktop and mobile, with no secrets exposed in frontend code.

08 / Agent-ready build brief

Give this brief and the repository to GPT-OSS, Antigravity, or another coding model. Ask it to implement one milestone at a time, run tests, and report changed files and results.

```
PROJECT: WEB FORGE AI Jury Scoring Portal

GOAL: Build a secure, minimal scoring website with Judge 1, Judge 2, and Admin. Rounds are Machine
Learning (ML) Model and Web Development. Each round has the five criteria listed in this document.
Maximum marks and weights are configurable. Do not invent a marking scheme.

STACK: Plain HTML/CSS/JavaScript frontend; Python Flask backend; dedicated Supabase PostgreSQL project,
separate from the quiz website. Use modular code, environment variables, schema/migration SQL, and README.

DESIGN: Palette #171717, #414141, #777777, #F2B99F, #FBE9E0, #FFFFFF, #F4F3F1. Minimal, accessible,
responsive. Square corners, thin dividers, restrained peach accents. No rounded cards, pill buttons,
glassmorphism, blur, translucent layers, or heavy gradients. No frontend framework without approval.

SECURITY: Never expose Supabase service-role credentials in frontend code. Enforce role checks on every
protected backend route. Derive judge identity from the authenticated session, never from a request field.
Judge 1 and Judge 2 may edit only their own scores. Admin alone manages judges, teams, criteria, results,
reopening, and exports. Validate IDs, relationships, score ranges, and evaluation state. Use secure sessions
or Supabase Auth, HTTPS, CSRF protection where applicable, and suitable RLS. Never store plain-text passwords.
Never commit .env files.

DATA: Tables for judges/profiles, participants, rounds, criteria, scores, evaluation_status, audit_log.
Use foreign keys, indexes, and unique constraints to prevent duplicate score/status records. Preserve history
when entities are deactivated. Freeze/version criteria after scoring starts.

WORKFLOW: Judge logs in, selects round and team, scores five criteria, saves draft or submits. Admin manages
judges, teams, criteria, sees both judges' results, reopens with audit logging, and exports CSV. Missing scores
are incomplete, not zero. Apply sum/average/weight formula only after owner confirms it.

PROCESS:
1. Inspect repository and propose changes before editing.
2. Implement schema and configuration.
3. Implement authentication and role checks.
4. Implement judge workflow and validation.
5. Implement Admin CRUD and reporting.
6. Add tests for permissions, mark bounds, duplicate records, locking, and calculations.
7. Run tests and fix failures. Never claim tests passed unless actually run.
8. Provide setup, environment variable NAMES (not values), deployment steps, and known limitations.
9. Ask before making assumptions that affect scoring, permissions, or data retention.
10. Keep code understandable for a student developer and avoid unnecessary abstractions.
```

Owner decisions required before implementation

1. Maximum marks and/or weights for each criterion.
2. Sum or average of both judges, and how weights apply.
3. Whether both judges score every team in both rounds.
4. Whether Admin can edit submitted marks or only reopen an evaluation.
5. Manual team entry or CSV/Excel import.

END / WEB FORGE AI JURY SCORING PORTAL